From Bits to Pixels

The Mathematical and Physical Foundations of elementary 3D Graphics

Jack Wetherell

Contents

1 Foundations: Numbers, Memory, and Representation **2**
1.1 Bits, Bytes, and Binary Storage . . . 2
1.2 Floating Point Representation (IEEE 754) . . . 2
1.3 From Raw Memory to Numerical Values . . . 3

2 Geometry Representation in Memory **4**
2.1 Vertices (Position Vectors) . . . 4
2.2 Triangles (Faces) . . . 4
2.3 Indexed vs Non-Indexed Geometry . . . 5
2.4 Meshes (Objects) . . . 5
2.5 Scenes as Collections of Objects . . . 6

3 Coordinate Systems and Spaces **6**
3.1 Local (Object) Space . . . 6
3.2 World Space . . . 7
3.3 Camera (View) Space . . . 7
3.4 Clip Space . . . 7
3.5 Normalized Device Coordinates (NDC) . . . 8
3.6 Screen Space . . . 8
3.7 Summary of Transform Pipeline . . . 9
3.8 Conceptual Interpretation . . . 9

4 Rasterization **9**
4.1 Triangle Setup . . . 9
4.2 Barycentric Coordinates . . . 10
4.3 Interpolation Across Surfaces . . . 10
4.4 Depth Testing (Z-Buffer) . . . 11
4.5 Summary . . . 11

5 Memory Layout of Graphics Data **11**
5.1 Vertex Buffers . . . 12
5.2 Index Buffers . . . 12
5.3 Transformation Storage . . . 12
5.4 Estimating Memory Usage . . . 13
5.5 Example: Pyramid . . . 14
5.6 Summary . . . 14

1 Foundations: Numbers, Memory, and Representation

1.1 Bits, Bytes, and Binary Storage

At the lowest level, all data in a computer is represented using binary digits.

- A **bit** is a single binary value:
$$b \in \{0, 1\} \tag{1}$$
- A **byte** consists of 8 bits:
$$1 \text{ byte} = 8 \text{ bits} \tag{2}$$

A sequence of bits can represent integers using positional notation in base 2:

$$N = \sum_{i=0}^{n-1} b_i \cdot 2^i \tag{3}$$

where:

- b_i is the i-th bit (0 or 1)
- $i = 0$ corresponds to the least significant bit (LSB)

Example:
$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10} \tag{4}$$

Memory is therefore a linear array of bytes:

$$\text{Memory} = [\text{byte}_0, \text{byte}_1, \ldots, \text{byte}_N] \tag{5}$$

Each byte has an address:

$$\text{Address} \in \mathbb{N} \tag{6}$$

1.2 Floating Point Representation (IEEE 754)

Real numbers are represented approximately using the IEEE 754 floating point format.

A 32-bit floating point number (single precision) is composed of:

- 1 bit for the sign (s)
- 8 bits for the exponent (e)
- 23 bits for the mantissa (fraction) (m)

The value represented is:

$$x = (-1)^s \cdot (1 + m) \cdot 2^{e-127} \tag{7}$$

where:

$$m = \sum_{i=1}^{23} m_i \cdot 2^{-i} \tag{8}$$

Interpretation:

- $(-1)^s$ determines the sign
- $(1 + m)$ is the normalized significand
- $e - 127$ is the unbiased exponent

Special Cases:

- $e = 0 \rightarrow$ denormalized numbers
- $e = 255 \rightarrow$ infinity or NaN

1.3 From Raw Memory to Numerical Values

All numerical values in a program are ultimately stored as sequences of bytes in memory.

A 32-bit float occupies 4 consecutive bytes:

$$\text{float} \rightarrow [b_0, b_1, b_2, b_3] \tag{9}$$

These bytes are interpreted according to the IEEE 754 structure:

$$\underbrace{s}_{1\text{ bit}} \quad \underbrace{e}_{8\text{ bits}} \quad \underbrace{m}_{23\text{ bits}} \tag{10}$$

Example: Representing $x = 7.98$ The decimal number 7.98 cannot be represented exactly in binary, so it is approximated.

Its IEEE 754 single-precision representation is:

$$x \approx 7.980000019073486 \tag{11}$$

Binary layout:

$$0\ 10000001\ 11111111001100110011010 \tag{12}$$

Breaking this down:

- Sign bit:
$$s = 0 \quad \text{(positive)} \tag{13}$$
- Exponent:
$$e = 10000001_2 = 129_{10} \tag{14}$$
$$e_{\text{unbiased}} = 129 - 127 = 2 \tag{15}$$
- Mantissa:
$$m = 1.11111111001100110011010_2 \tag{16}$$

Reconstructing the value:

$$x = (-1)^0 \cdot (1 + m_{\text{fraction}}) \cdot 2^2 \tag{17}$$

Key Insight: There is no intrinsic meaning in memory—only interpretation.

$$\text{Bits} \xrightarrow{\text{interpretation}} \text{Number} \tag{18}$$

The same sequence of bits may represent:

- an integer
- a floating point number
- part of a vertex
- or any structured data

Conclusion: All higher-level constructs (vectors, matrices, geometry) are built from:

$$\text{Bits} \to \text{Bytes} \to \text{Floats} \to \text{Mathematical Objects} \tag{19}$$

2 Geometry Representation in Memory

2.1 Vertices (Position Vectors)

A **vertex** represents a point in 3D space.

In its simplest form, a vertex consists of three floating-point values:

$$\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \in \mathbb{R}^3 \tag{20}$$

For transformation purposes, vertices are often expressed in homogeneous coordinates:

$$\mathbf{v}_h = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \in \mathbb{R}^4 \tag{21}$$

Memory Representation: Each coordinate is a 32-bit float:

$$\text{vertex} = [x, y, z] \tag{22}$$

$$\text{size of vertex} = 3 \times 4 \text{ bytes} = 12 \text{ bytes} \tag{23}$$

Thus, a vertex buffer is:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{24}$$

2.2 Triangles (Faces)

The fundamental rendering primitive is the **triangle**.

A triangle is defined by three vertices:

$$\triangle = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2) \tag{25}$$

Geometrically, any point inside the triangle can be expressed using barycentric coordinates:

$$\mathbf{p} = \alpha\mathbf{v}_0 + \beta\mathbf{v}_1 + \gamma\mathbf{v}_2 \tag{26}$$

subject to:

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0 \tag{27}$$

Memory Representation (Non-Indexed): Each triangle stores three full vertices:

$$\text{triangle size} = 3 \times 12 \text{ bytes} = 36 \text{ bytes} \tag{28}$$

2.3 Indexed vs Non-Indexed Geometry

Non-Indexed Representation: Each triangle explicitly stores its vertices:

$$T = [\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2] \tag{29}$$

This leads to duplication of shared vertices.

Indexed Representation: Vertices are stored once in a vertex buffer:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{30}$$

Triangles are defined using indices:

$$F = [(i_0, i_1, i_2), (i_3, i_4, i_5), (i_6, i_7, i_8)] \tag{31}$$

Where winding order is used to specify face direction.
Each index refers to a vertex:

$$\triangle_k = (\mathbf{v}_{i_a}, \mathbf{v}_{i_b}, \mathbf{v}_{i_c}) \tag{32}$$

Memory Efficiency: Let:

$$n_v = \text{number of vertices}, \quad n_t = \text{number of triangles} \tag{33}$$

Then:

$$\text{non-indexed size} = 36 \cdot n_t \tag{34}$$

$$\text{indexed size} = 12 \cdot n_v + 12 \cdot n_t \tag{35}$$

(assuming 32-bit indices)

2.4 Meshes (Objects)

A **mesh** (object) consists of:

- A vertex buffer V
- An index buffer F

$$\mathcal{M} = (V, F) \tag{36}$$

The mesh is defined in **local (object) space**, meaning:

$$\mathbf{v}_{local} \in \mathbb{R}^3 \tag{37}$$

All vertices are expressed relative to the object's origin:

$$\mathbf{o}_{local} = (0, 0, 0) \tag{38}$$

2.5 Scenes as Collections of Objects

A **scene** consists of multiple objects:

$$\mathcal{S} = \{\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_k\} \tag{39}$$

Each object has an associated transformation:

$$M_i \in \mathbb{R}^{4\times 4} \tag{40}$$

which maps local coordinates to world coordinates:

$$\mathbf{v}_{world} = M_i \mathbf{v}_{local} \tag{41}$$

Global Interpretation: All objects share a common coordinate system:

$$\mathbf{v}_{world} \in \mathbb{R}^3 \tag{42}$$

Thus, the scene becomes a unified collection of geometry in world space.

Final Structure in Memory:

$$\text{Scene} = \{(V_1, F_1, M_1), (V_2, F_2, M_2), \ldots\} \tag{43}$$

Each component ultimately reduces to:

$$\text{Bytes} \to \text{Floats} \to \text{Vertices} \to \text{Triangles} \to \text{Meshes} \to \text{Scene} \tag{44}$$

3 Coordinate Systems and Spaces

In computer graphics, geometry is transformed through a sequence of coordinate systems. Each space provides a different frame of reference for interpreting vertex positions.

$$\mathbf{v}_{local} \to \mathbf{v}_{world} \to \mathbf{v}_{camera} \to \mathbf{v}_{clip} \to \mathbf{v}_{ndc} \to \mathbf{v}_{screen} \tag{45}$$

3.1 Local (Object) Space

Local space (or object space) is the coordinate system in which a mesh is originally defined.

$$\mathbf{v}_{local} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \tag{46}$$

Key properties:

- Origin is the object's center:
$$\mathbf{o}_{local} = (0, 0, 0) \tag{47}$$
- Basis vectors define the object's orientation:
$$\{\mathbf{e}_x^{local}, \mathbf{e}_y^{local}, \mathbf{e}_z^{local}\} \tag{48}$$

Vertices are stored in this space in memory.

3.2 World Space

World space is a global coordinate system shared by all objects in the scene.

Each object is placed into world space using a model transformation:

$$\mathbf{v}_{world} = M_{model}\mathbf{v}_{local} \tag{49}$$

where:

$$M_{model} = T \cdot R \cdot S \tag{50}$$

Explicitly:

$$M_{model} = \begin{bmatrix} r_{11}S_x & r_{12}S_y & r_{13}S_z & t_x \\ r_{21}S_x & r_{22}S_y & r_{23}S_z & t_y \\ r_{31}S_x & r_{32}S_y & r_{33}S_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{51}$$

Key properties:

- Common reference frame for all objects
- Fixed global basis:

$$\{\mathbf{e}_x^{world}, \mathbf{e}_y^{world}, \mathbf{e}_z^{world}\} \tag{52}$$

3.3 Camera (View) Space

Camera space (or view space) represents coordinates relative to the camera.

Instead of moving the camera, we transform the world:

$$\mathbf{v}_{camera} = M_{view}\mathbf{v}_{world} \tag{53}$$

The view matrix is the inverse of the camera transform:

$$M_{view} = (T_{cam}R_{cam})^{-1} \tag{54}$$

If the camera has position $\mathbf{c}$ and rotation R_{cam}:

$$M_{view} = \begin{bmatrix} R_{cam}^T & -R_{cam}^T\mathbf{c} \\ 0 & 1 \end{bmatrix} \tag{55}$$

Key properties:

- Camera is located at origin:

$$\mathbf{o}_{camera} = (0, 0, 0) \tag{56}$$

- Viewing direction typically along negative z-axis

3.4 Clip Space

Clip space is obtained by applying the projection matrix:

$$\mathbf{v}_{clip} = M_{proj}\mathbf{v}_{camera} \tag{57}$$

A standard perspective projection matrix is:

$$M_{proj} = \begin{bmatrix} \frac{1}{a\tan(\frac{\text{fov}}{2})} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\frac{\text{fov}}{2})} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix} \tag{58}$$

where:

$$a = \text{aspect ratio}, \quad n = \text{near plane}, \quad f = \text{far plane} \quad \text{fov} = \text{field of view} \tag{59}$$

Vertices in clip space are 4D:

$$\mathbf{v}_{clip} = \begin{bmatrix} x_c \\ y_c \\ z_c \\ w_c \end{bmatrix} \tag{60}$$

3.5 Normalized Device Coordinates (NDC)

To obtain normalized device coordinates, perform the perspective divide:

$$\mathbf{v}_{ndc} = \begin{bmatrix} x_c/w_c \\ y_c/w_c \\ z_c/w_c \end{bmatrix} \tag{61}$$

This maps coordinates into a canonical cube:

$$x, y, z \in [-1, 1] \tag{62}$$

Key properties:

- Removes perspective distortion via division by w
- Enables uniform clipping and rasterization

3.6 Screen Space

Screen space maps normalized coordinates to pixel coordinates.

For a viewport of width W and height H:

$$x_{screen} = \frac{W}{2}(x_{ndc} + 1) \tag{63}$$

$$y_{screen} = \frac{H}{2}(1 - y_{ndc}) \tag{64}$$

$$z_{screen} = \frac{1}{2}(z_{ndc} + 1) \tag{65}$$

Final screen coordinate:

$$\mathbf{v}_{screen} = \begin{bmatrix} x_{screen} \\ y_{screen} \\ z_{screen} \end{bmatrix} \tag{66}$$

3.7 Summary of Transform Pipeline

The full transformation pipeline is:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{67}$$

Followed by:

$$\mathbf{v}_{ndc} = \frac{1}{w}\mathbf{v}_{clip} \tag{68}$$

$$\mathbf{v}_{screen} = \text{Viewport}(\mathbf{v}_{ndc}) \tag{69}$$

3.8 Conceptual Interpretation

Each transformation corresponds to a change of coordinate system:

$$\text{Local} \to \text{World} \to \text{Camera} \to \text{Clip} \to \text{Screen} \tag{70}$$

At every stage:

- Coordinates are expressed relative to a specific basis
- Matrix multiplication re-expresses vectors in a new basis

Thus, the entire graphics pipeline can be viewed as a sequence of basis transformations applied to vertex data.

4 Rasterization

Rasterization is the process of converting geometric primitives (triangles) into discrete pixels on the screen.

Given vertices in screen space:

$$\mathbf{v}_0 = (x_0, y_0, z_0), \quad \mathbf{v}_1 = (x_1, y_1, z_1), \quad \mathbf{v}_2 = (x_2, y_2, z_2) \tag{71}$$

the goal is to determine which pixels lie inside the triangle and compute their attributes.

4.1 Triangle Setup

A triangle is defined by three vertices in screen space:

$$\triangle = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2) \tag{72}$$

We define edge functions to determine whether a point lies inside the triangle.

For a point $\mathbf{p} = (x, y)$, define:

$$E_{ij}(\mathbf{p}) = (x - x_i)(y_j - y_i) - (y - y_i)(x_j - x_i) \tag{73}$$

A point is inside the triangle if:

$$E_{01}(\mathbf{p}) \geq 0, \quad E_{12}(\mathbf{p}) \geq 0, \quad E_{20}(\mathbf{p}) \geq 0 \tag{74}$$

(for a consistently wound triangle)

Interpretation: Each edge function represents a signed area test.

4.2 Barycentric Coordinates

Any point inside the triangle can be expressed as a weighted combination of the vertices:

$$\mathbf{p} = \alpha \mathbf{v}_0 + \beta \mathbf{v}_1 + \gamma \mathbf{v}_2 \tag{75}$$

with:

$$\alpha + \beta + \gamma = 1 \tag{76}$$

The barycentric coordinates can be computed using areas:

$$\alpha = \frac{A(\mathbf{p}, \mathbf{v}_1, \mathbf{v}_2)}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{77}$$

$$\beta = \frac{A(\mathbf{v}_0, \mathbf{p}, \mathbf{v}_2)}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{78}$$

$$\gamma = \frac{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{p})}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{79}$$

where:

$$A(\mathbf{a}, \mathbf{b}, \mathbf{c}) = \frac{1}{2} \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} \tag{80}$$

4.3 Interpolation Across Surfaces

Once barycentric coordinates are known, any vertex attribute can be interpolated.

Let q be a per-vertex attribute (e.g. depth, color, texture coordinate):

$$q_0, \quad q_1, \quad q_2 \tag{81}$$

Then:

$$q(\mathbf{p}) = \alpha q_0 + \beta q_1 + \gamma q_2 \tag{82}$$

Important: Perspective-Correct Interpolation Due to perspective projection, interpolation must be corrected using w:

$$q' = \frac{q}{w} \tag{83}$$

Interpolate:

$$q'(\mathbf{p}) = \alpha q'_0 + \beta q'_1 + \gamma q'_2 \tag{84}$$

Then recover:

$$q(\mathbf{p}) = \frac{q'(\mathbf{p})}{\alpha \frac{1}{w_0} + \beta \frac{1}{w_1} + \gamma \frac{1}{w_2}} \tag{85}$$

4.4 Depth Testing (Z-Buffer)

Each pixel stores a depth value:

$$z_{\text{buffer}}(x, y) \tag{86}$$

For a candidate fragment with depth z_{new}:

$$z_{\text{new}} < z_{\text{buffer}}(x, y) \Rightarrow \text{fragment is visible} \tag{87}$$

If true:

$$z_{\text{buffer}}(x, y) \leftarrow z_{\text{new}} \tag{88}$$

$$\text{color}(x, y) \leftarrow \text{fragment color} \tag{89}$$

Otherwise, the fragment is discarded.

Depth Range: After projection:

$$z \in [0, 1] \tag{90}$$

Non-linearity: Depth is non-linear with respect to camera distance:

$$z \sim \frac{1}{z_{camera}} \tag{91}$$

This provides higher precision near the camera.

4.5 Summary

Rasterization converts triangles into pixels through:

$$\text{Triangle} \rightarrow \text{Fragments} \rightarrow \text{Pixels} \tag{92}$$

Key steps:

- Determine pixel coverage (edge functions)
- Compute barycentric coordinates
- Interpolate attributes
- Perform depth testing

Thus, rasterization bridges continuous geometry and discrete image representation.

5 Memory Layout of Graphics Data

All geometric data in a graphics pipeline is ultimately stored as structured sequences of bytes in memory (CPU RAM or GPU VRAM). These structures are designed for efficient access and parallel processing.

5.1 Vertex Buffers

A **vertex buffer** stores a sequence of vertices:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_{n-1}] \tag{93}$$

Each vertex typically contains multiple attributes:

$$\mathbf{v}_i = \Big[x_i, y_i, z_i, w_i,\ n_x, n_y, n_z,\ u, v,\ \ldots\Big] \tag{94}$$

Memory Layout: Each attribute is stored as a float (4 bytes):

$$\text{size of vertex} = k \cdot 4 \text{ bytes} \tag{95}$$

where k is the number of float components.

Example (Position Only):

$$\mathbf{v}_i = (x, y, z) \tag{96}$$

$$\text{vertex size} = 3 \times 4 = 12 \text{ bytes} \tag{97}$$

Contiguous Storage:

$$\text{Vertex Buffer} = [x_0, y_0, z_0,\ x_1, y_1, z_1,\ \ldots] \tag{98}$$

5.2 Index Buffers

An **index buffer** defines how vertices form triangles.

$$F = [(i_0, i_1, i_2), (i_3, i_4, i_5), \ldots] \tag{99}$$

Each index refers to a vertex in the vertex buffer:

$$\triangle_k = (\mathbf{v}_{i_a}, \mathbf{v}_{i_b}, \mathbf{v}_{i_c}) \tag{100}$$

Memory Representation: Indices are typically stored as integers:

$$\text{index size} = \begin{cases} 2 \text{ bytes (16-bit)} \\ 4 \text{ bytes (32-bit)} \end{cases} \tag{101}$$

Example:

$$F = [0, 1, 2,\ 2, 3, 0] \tag{102}$$

This represents two triangles sharing vertices.

5.3 Transformation Storage

Each object in a scene has an associated transformation matrix:

$$M \in \mathbb{R}^{4\times 4} \tag{103}$$

Matrix Layout:

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix} \tag{104}$$

Memory Size:

$$\text{matrix size} = 16 \times 4 = 64 \text{ bytes} \tag{105}$$

Storage Format: Matrices are stored as a flat array:

$$[m_{11}, m_{12}, \ldots, m_{44}] \tag{106}$$

Note: Different APIs may use:

- Row-major order
- Column-major order

5.4 Estimating Memory Usage

Let:

$$n_v = \text{number of vertices} \tag{107}$$

$$n_t = \text{number of triangles} \tag{108}$$

$$k = \text{floats per vertex} \tag{109}$$

Vertex Buffer Size:

$$\text{size}_V = n_v \cdot k \cdot 4 \tag{110}$$

Index Buffer Size: Each triangle uses 3 indices:

$$\text{size}_F = n_t \cdot 3 \cdot s_i \tag{111}$$

where:

$$s_i \in \{2, 4\} \text{ bytes} \tag{112}$$

Transformation Size: For n_o objects:

$$\text{size}_M = n_o \cdot 64 \tag{113}$$

Total Memory:

$$\text{Total} = \text{size}_V + \text{size}_F + \text{size}_M \tag{114}$$

5.5 Example: Pyramid

Consider a pyramid with:

- $n_v = 5$ vertices
- $n_t = 6$ triangles
- $k = 3$ (position only)

Vertex Memory:

$$\text{size}_V = 5 \cdot 3 \cdot 4 = 60 \text{ bytes} \tag{115}$$

Index Memory (32-bit):

$$\text{size}_F = 6 \cdot 3 \cdot 4 = 72 \text{ bytes} \tag{116}$$

Transform:

$$\text{size}_M = 64 \text{ bytes} \tag{117}$$

Total:

$$\text{Total} = 60 + 72 + 64 = 196 \text{ bytes} \tag{118}$$

5.6 Summary

Graphics data in memory is structured as:

$$\text{Scene} = \{(V_i, F_i, M_i)\}_{i=1}^{n_o} \tag{119}$$

Each component is stored as contiguous bytes:

$$\text{Bytes} \to \text{Floats} \to \text{Vertices} \to \text{Triangles} \to \text{Objects} \tag{120}$$

Efficient layout enables:

- Coalesced memory access on GPU
- Parallel processing of vertices
- High throughput rendering